

Chapter

Python 3 Decorators

First-Class Functions, Closures, and the Decorator Pattern in Practice

Table of Contents

Table of Contents	2
Abstract	4
Part I — Theoretical Foundations	5
1. Functions as First-Class Objects	5
2. Higher-Order Functions	5
3. Closures	5
4. Decorators: Definition and Syntax	6
5. Decorator Execution Flow	6
6. Generalizing Decorators with *args and **kwargs	7
7. functools.wraps and Metadata Preservation	7
8. Standard Library Decorators	8
8.1 @classmethod	8
8.2 @property	8
8.3 @lru_cache	8
9. Canonical Custom Decorator Patterns	9
10. Decorator Benefits and Trade-offs	9
Part II — Practical Implementation and Annotated Code	11
11. First-Class Functions and Higher-Order Functions	11
11.1 Assigning a Function to a New Name	11
11.2 Passing a Function as an Argument	11
12. Closures	11
12.1 An Inner Function Capturing an Outer Variable	11
13. Building a Decorator from First Principles	12
13.1 A Minimal Decorator	12
13.2 A Practical Variant: Formatting a Message	12
14. Generalizing Decorators for Arbitrary Signatures	13
14.1 Forwarding Arguments with *args and **kwargs	13
14.2 Preserving Metadata with functools.wraps	14
15. Standard Library Decorators in Use	14
15.1 Memoization with lru_cache	14
15.2 Class-Level Behavior with classmethod	14
15.3 Computed Attributes with property	15
15.4 Observing the Cache Directly	15

16. Real-World Decorator Templates	16
16.1 A Timing Decorator	16
16.2 A Logging Decorator	16
Summary	16

Abstract

Decorators are Python's idiomatic mechanism for separating cross-cutting concerns — logging, timing, caching, validation, access control — from the business logic they augment, without modifying the augmented function's source. This capability rests on two more primitive language properties: functions as first-class objects, and closures, which allow an inner function to retain access to variables from an enclosing scope after that scope has returned. This chapter develops the theory of first-class functions, higher-order functions, and closures as the foundation for understanding what a decorator actually is at the language-semantic level, then builds up the decorator pattern itself — from a minimal argument-less wrapper, through `*args/**kwargs` generalization, to `functools.wraps` metadata preservation and the standard library's own decorators (`@classmethod`, `@property`, `@lru_cache`). Part I presents the underlying theory; Part II operationalizes it through annotated code listings, each followed by a cell-level explanation of intent and mechanism.

Part I — Theoretical Foundations

1. Functions as First-Class Objects

In Python, a function is an object like any other: an instance of the function type, allocated on the heap, bindable to a name, storable in a data structure, passable as an argument, and returnable as a value. This property — functions as first-class citizens of the language — is the foundational fact from which decorators, callbacks, strategy-pattern implementations, and functional-programming idioms in Python all derive. It implies specifically that the statement `def greet(): ...` does not create a special, restricted kind of entity; it creates an ordinary object and binds the name `greet` to it, in exactly the same sense that `x = 5` binds the name `x` to an integer object. Consequently, a second name can be bound to the same function object (`message = greet`), and invoking that second name (`message()`) calls the identical underlying function — the name used to call a function is merely a reference to it, not part of the function's identity.

This matters for decorators specifically because a decorator's entire mechanism depends on functions being values that can be received as parameters and produced as return values; without first-class function status, the pattern described in this chapter would not be expressible in the language at all.

2. Higher-Order Functions

A higher-order function is a function that operates on other functions — by accepting one or more functions as arguments, by returning a function as its result, or both. This is a direct consequence of first-class function status: if functions are ordinary values, nothing distinguishes a function-typed parameter or return value from a parameter or return value of any other type. Higher-order functions underlie a wide range of idiomatic Python beyond decorators — `map()`, `filter()`, `sorted()` with a key function, and callback-based APIs all accept a function as an argument so that variable behavior can be supplied by the caller without modifying the higher-order function itself.

A decorator is, by definition, a higher-order function: it accepts a function as its argument and returns a (typically different) function as its result. This definitional fact is what makes decorators a strict subset of the higher-order function concept rather than an independent language feature — the `@` syntax introduced in §4 is purely sugar over an ordinary higher-order function call.

3. Closures

A closure arises when an inner function references a variable from its immediately enclosing (non-global) scope, and that inner function is then returned from, or otherwise outlives, the enclosing function's execution. Ordinarily, a local variable's storage is reclaimed once its containing function returns; a closure is the mechanism by which the inner function retains a live reference to that variable's binding — via a cell object capturing the enclosing scope's variable — so the variable continues to exist and remain accessible for as long as the inner function itself exists, even though the outer function's stack frame has long since been popped.

Closures are not a decorator-specific feature; they are a general property of nested function definitions in Python (and in any language with lexical scoping and first-class functions). Their relevance to decorators is structural: every decorator implementation in this chapter defines an inner wrapper function that must retain a reference to the original, undecorated function passed into the decorator — and that retention is precisely what a closure provides. Without closures, the wrapper would have no way to remember which function it is supposed to wrap once the decorator function itself has returned. A function object's `__closure__` attribute exposes the captured cell objects, making the otherwise-invisible captured state inspectable.

4. Decorators: Definition and Syntax

A decorator is a callable that accepts a function (or, more generally, a class) and returns a callable intended to be used as a drop-in replacement for it — typically a wrapper that executes additional behavior before and/or after delegating to the original. Because the original function's source code is never altered, decorators provide a means of augmenting behavior that is fully separable from, and orthogonal to, the function's own implementation, which is the core motivation for using them: cross-cutting concerns (logging, timing, authorization, memoization) can be added, removed, or composed without touching the decorated function's body.

The `@decorator` syntax placed immediately above a function definition is syntactic sugar for re-assigning the defined name to the decorator's return value: writing `@decorator` above `def greet(): ...` is exactly equivalent to writing `def greet(): ...` followed by `greet = decorator(greet)`. This equivalence is the single most important fact for understanding decorator semantics, since it makes explicit that decoration is simply a name rebinding performed automatically by the interpreter at function-definition time, using the ordinary function-call and assignment machinery already described in §§1–2.

5. Decorator Execution Flow

At the moment a decorated function definition is executed (i.e., at module import / class body execution time, not at call time), the following sequence occurs: the undecorated function object is constructed first; it is passed as the sole argument to the decorator function; the decorator constructs and returns a wrapper function (via closure, retaining a reference to the original); and the original name is rebound to point to that wrapper rather than to the original function object. From this point forward, every call using the decorated function's name actually invokes the wrapper, which in turn calls the original function — via its closure-captured reference — at whatever point in the wrapper's body the implementer chose to place that call, surrounding it with whatever additional behavior the wrapper defines before and after.

This sequencing has a non-obvious but important consequence: decoration is a one-time, definition-time cost (constructing the wrapper), while the wrapper's extra behavior is a recurring, per-call cost (executed on every invocation of the decorated name) — a distinction relevant when reasoning about a decorator's performance impact in a hot code path.

6. Generalizing Decorators with `*args` and `**kwargs`

A wrapper function written to take no parameters can only decorate functions that themselves take no parameters, since the wrapper's call to the wrapped function must supply whatever arguments that function expects. Writing the wrapper's signature as `def wrapper(*args, **kwargs)` instead allows it to accept an arbitrary number of positional and keyword arguments, collecting them into a tuple and a dict respectively, and forwarding them unchanged via `func(*args, **kwargs)`. This generalization is what makes a single decorator implementation reusable across functions of differing, even varying, arity and signature, rather than requiring a bespoke wrapper signature per decorated function — a property essential to any decorator intended for general-purpose use (logging, timing, caching) rather than for one specific function.

7. `functools.wraps` and Metadata Preservation

Because decoration rebinds the original name to the wrapper function object, introspection of the decorated name — `__name__`, `__doc__`, `__module__`, and similar dunder attributes — will, by default, report the wrapper's own metadata rather than the original function's, since these attributes belong to whichever object the name currently references. This is usually undesirable: tools that rely on a function's name or docstring for documentation generation, debugging, logging output, or registry lookups will see

misleading information (e.g., every decorated function appearing to be named 'wrapper').

`functools.wraps(func)`, applied as a decorator on the wrapper function itself (immediately above `def wrapper(...): ...`), copies the relevant metadata attributes from `func` onto `wrapper` before the wrapper is returned, so that the final decorated name continues to expose the original function's identity under introspection while still actually executing the wrapper's logic. This is considered a required, not optional, convention for any custom decorator intended for real use, since omitting it silently degrades introspectability and tooling compatibility without producing any visible error.

8. Standard Library Decorators

Python's standard library and built-in namespace define several decorators that apply the same underlying mechanism (§§4–5) to specific, common needs, demonstrating that the decorator pattern is not solely a user-level idiom but a core part of idiomatic Python itself.

8.1 `@classmethod`

`@classmethod` transforms a method so that it receives the class itself (conventionally named `cls`) as its first parameter, rather than a particular instance (`self`). This makes the method's behavior bound to the class as a whole rather than to any one object, allowing it to read or mutate class-level attributes (shared across all instances) and making it the idiomatic mechanism for defining alternative constructors — factory methods that build and return an instance via logic distinct from `__init__`, while still being invocable on the class without first creating an instance.

8.2 `@property`

`@property` converts an instance method into something accessed with attribute syntax rather than call syntax — `obj.name` instead of `obj.name()` — while still executing arbitrary method logic on each access. This supports encapsulation: a class can expose what appears to callers as a plain attribute while internally computing the value, validating it, or deriving it from other state, and can later change that internal implementation without breaking the external attribute-style interface. It specifically avoids the explicit `get_x()/set_x()` getter/setter boilerplate common in languages without this mechanism, producing cleaner call-site code while preserving the ability to add validation or computed logic transparently.

8.3 `@lru_cache`

`functools.lru_cache` applies the decorator pattern specifically to memoization: it wraps a function so that the result for any previously-seen combination of arguments is retrieved from an internal cache rather than recomputed, evicting the least-recently-used entries once a configured maxsize is exceeded. As discussed in the companion chapter on performance, this is especially impactful for recursive functions with overlapping subproblems (the canonical example being naive recursive Fibonacci), where it converts exponential-time recomputation into linear-time computation by guaranteeing each unique input is computed at most once. `lru_cache` itself is a clear, practical illustration of §7's metadata concern resolved correctly: the standard library's implementation already applies `functools.wraps` internally, so a function decorated with `@lru_cache` retains its original `__name__` and `__doc__` automatically.

9. Canonical Custom Decorator Patterns

Two decorator implementations recur constantly in real-world Python codebases and serve as the template for most others: the timing decorator, which wraps a function to measure and report its execution duration, and the logging decorator, which wraps a function to record the fact, and optionally the arguments, of each call. Both follow the identical structural template established in §§4–7 — a generalized `(*args, **kwargs)` wrapper, decorated with `@wraps(func)` to preserve metadata, surrounding a single delegating call to the original function — differing only in what observable side effect is performed before and after that delegating call. This structural uniformity is precisely the benefit decorators are designed to provide: a new cross-cutting concern can be added to a codebase by writing one such wrapper once and applying it via `@` syntax to as many functions as required, with zero modification to any of those functions' internal logic.

10. Decorator Benefits and Trade-offs

- Separation of concerns: business logic in the decorated function remains uncluttered by cross-cutting infrastructure code (logging, timing, validation).
- Code reuse: a single decorator implementation can be applied to any number of functions, eliminating duplicated boilerplate that would otherwise be repeated inside each function body.
- Composability: multiple decorators can be stacked on a single function, each contributing one orthogonal behavior, applied in the order listed (innermost decorator nearest the function executes first when wrapping, and its wrapper runs last when calling, due to the nesting this produces).

- Non-invasiveness: behavior can be added to, or removed from, a function without editing that function's own source, reducing the risk of introducing defects into already-validated logic.

The trade-offs are the same ones that apply to any indirection-introducing abstraction: each layer of decoration adds a function call's worth of overhead per invocation, can complicate debugging since a stack trace passes through the wrapper before reaching the original function, and — if `functools.wraps` is omitted — silently degrades introspection as described in §7. None of these are reasons to avoid decorators generally, but they are reasons to apply the pattern deliberately rather than reflexively, consistent with the broader optimization-and-abstraction cost-benefit principle introduced in the performance chapter.

Part II — Practical Implementation and Annotated Code

Each listing below corresponds to a concept introduced in Part I and is treated as an independent notebook cell. Explanations are given at the what/why level for the cell as a whole, consistent with the convention established in the companion performance chapter.

11. First-Class Functions and Higher-Order Functions

11.1 Assigning a Function to a New Name

```
def greet():  
    print("Hello")  
  
message = greet  
message()
```

Cell explanation: `def greet()` creates a function object and binds the name `greet` to it. The assignment `message = greet` does not copy or re-execute the function; it binds a second name, `message`, to the same underlying function object — demonstrating directly that a function is an ordinary, reference-assignable value. Calling `message()` therefore invokes exactly the same code as `greet()` would, which is the concrete illustration of the first-class-function claim in Part I §1.

11.2 Passing a Function as an Argument

```
def execute(func):  
    func()  
  
def hello():  
    print("Hello")  
  
execute(hello)
```

Cell explanation: `execute` is defined to accept a parameter `func` and call it as `execute's` own body executes — making `execute` a higher-order function per Part I §2, since it operates on a function passed to it rather than on data. Passing `hello` (the function object itself, not `hello()`, which would pass `hello's` return value) means `execute(hello)` ultimately triggers `hello's` print statement, despite `execute's` own body containing no reference to printing at all. This is the minimal pattern that every decorator generalizes.

12. Closures

12.1 An Inner Function Capturing an Outer Variable

```
def outer():
```

```
message = "Hello"
def inner():
    print(message)
    return inner

func = outer()
func()
```

Cell explanation: `outer` defines a local variable `message` and a nested function `inner` that references it, then returns `inner` itself (not `inner()`) — returning the function object, not the result of calling it. By the time `func = outer()` completes, `outer`'s stack frame has been popped, yet calling `func()` still successfully prints "Hello". This is only possible because `inner` forms a closure over `message`: Python detects that `inner` references a variable from its enclosing scope and keeps that variable's binding alive, attached to the function object, for as long as `inner` itself exists — directly demonstrating the closure mechanism described in Part I §3 that every decorator implementation in this chapter depends on.

13. Building a Decorator from First Principles

13.1 A Minimal Decorator

```
def decorator(func):
    def wrapper():
        print("Before")
        func()
        print("After")
    return wrapper

@decorator
def greet():
    print("Hello")

greet()
```

Cell explanation: `decorator` accepts a function `func` and defines `wrapper`, a closure that retains a reference to `func`, prints "Before", calls `func()`, then prints "After"; `decorator` then returns `wrapper` rather than calling it. The `@decorator` line above `def greet(): ...` is exactly equivalent to writing `greet = decorator(greet)` after the function definition, per Part I §4 — so after this code runs, the name `greet` no longer refers to the original function but to `wrapper`, with the original `greet` retained only via `wrapper`'s closure. Calling `greet()` therefore executes `wrapper`'s full Before/Hello/After sequence, not just the original `print("Hello")`.

13.2 A Practical Variant: Formatting a Message

```
def email_decorator(func):
```

```

def wrapper():
    print("Dear interns,")
    func()
    print("Best regards,")
    print("your new boss")
    return wrapper

@email_decorator
def greeting_message():
    print("Welcome to your new job!")

greeting_message()

```

Cell explanation: Structurally identical to §13.1, but here the surrounding behavior (a salutation before, a signature after) gives the pattern a concrete, realistic purpose: `greeting_message`'s own body is concerned only with its core message, while `email_decorator` is solely responsible for the letter-formatting boilerplate around it. This is the first listing in the chapter that demonstrates the actual motivating use case for decorators — separating a function's core concern from formatting/infrastructure concerns that surround it, per Part I §10 — rather than a syntax-focused minimal example.

14. Generalizing Decorators for Arbitrary Signatures

14.1 Forwarding Arguments with **args* and ***kwargs*

```

def logger(func):
    def wrapper(*args, **kwargs):
        print("Function called")
        return func(*args, **kwargs)
    return wrapper

@logger
def add(a, b):
    return a + b

print(add(5, 3))

```

Cell explanation: `wrapper`'s signature (`*args, **kwargs`) accepts any combination of positional and keyword arguments, collecting positional ones into the tuple `args` and keyword ones into the dict `kwargs`; `func(*args, **kwargs)` then unpacks them back out when forwarding the call to the original function. Because `add(5, 3)` supplies two positional arguments, `args` becomes `(5, 3)` and `kwargs` becomes `{}`, and `func(*args, **kwargs)` becomes equivalent to `add(5, 3)` — correctly reproducing the original call regardless of `add`'s specific signature. This is the generalization described in Part I §6 that makes `logger` reusable on functions of arbitrary arity, in contrast to the no-parameter wrapper in §13.

14.2 Preserving Metadata with `functools.wraps`

```
from functools import wraps

def logger(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print("Running function")
        return func(*args, **kwargs)
    return wrapper
```

Cell explanation: This is the same logger structure as §14.1, with one addition: `@wraps(func)` applied directly above wrapper's own definition. `wraps` is itself a decorator (taking `func` as a configuration argument and returning a decorator specifically for wrapper), and its effect is to copy `func`'s `__name__`, `__doc__`, and related metadata onto wrapper before wrapper is returned. Without this line, any function decorated with this logger would report its name as 'wrapper' under introspection (e.g., `decorated_func.__name__`) instead of its real name — the failure mode described in Part I §7 that `@wraps` exists specifically to prevent.

15. Standard Library Decorators in Use

15.1 Memoization with `lru_cache`

```
from functools import lru_cache

@lru_cache(maxsize=100)
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
```

Cell explanation: `@lru_cache(maxsize=100)` wraps `fibonacci` so that the result for any `n` already computed in a prior call is returned directly from cache rather than recomputed, bounding the cache to the 100 most recently used results. Because naive recursive Fibonacci recomputes the same subproblems repeatedly without memoization, this decorator alone changes the function's effective time complexity from exponential to linear — a direct, practical instance of the standard-library decorator pattern described in Part I §8.3, requiring no change to `fibonacci`'s own logic beyond the one decorator line.

15.2 Class-Level Behavior with `classmethod`

```
class Student:
    count = 0

    @classmethod
```

```
def increase_count(cls):
    cls.count += 1
```

Cell explanation: `@classmethod` transforms `increase_count` so that, regardless of how it is called, it receives the class itself (bound to the parameter `cls`) rather than a particular instance. `cls.count += 1` therefore mutates the class-level attribute `count` shared by all instances of `Student`, rather than creating or mutating a per-instance attribute — demonstrating the class-scoped behavior described in Part I §8.1, and illustrating why `classmethod` is the natural choice whenever a method's purpose is to read or modify state belonging to the class as a whole.

15.3 Computed Attributes with *property*

```
class Person:
    def __init__(self, name):
        self._name = name

    @property
    def name(self):
        return self._name
```

Cell explanation: The constructor stores the given name in a 'private-by-convention' attribute `_name`; the `name` method, decorated with `@property`, allows callers to read `person.name` using plain attribute syntax even though `name` is implemented as a method body that executes on every access. This satisfies Part I §8.2's encapsulation rationale: external code interacts with a `Person` instance as though `name` were a simple public attribute, while the class retains full control over how that value is actually stored and could, without changing the external interface, add validation, derive the value dynamically, or pair this getter with a corresponding `@name.setter` for controlled mutation.

15.4 Observing the Cache Directly

```
from functools import lru_cache

@lru_cache()
def square(n):
    print("Calculating...")
    return n * n
```

Cell explanation: Unlike §15.1, this listing includes a visible side effect (`print("Calculating...")`) specifically so the caching behavior can be observed empirically: calling `square(4)` twice in succession will print 'Calculating...' only on the first call, with the second call returning the cached result silently — making the cache's effect directly verifiable rather than merely asserted, which is a useful diagnostic pattern when first confirming that a memoized function is behaving as intended.

16. Real-World Decorator Templates

16.1 A Timing Decorator

```
import time
from functools import wraps

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print("Execution Time:", end - start)
        return result
    return wrapper
```

Cell explanation: This combines every element introduced so far: a generalized (*args, **kwargs) signature (§14.1) so timer can wrap any function regardless of arity, @wraps(func) to preserve metadata (§14.2), and a wrapper body that records a timestamp before delegating to func, records a second timestamp after, and reports the difference — while still capturing and returning func's actual result so that decorating a function with @timer does not change what callers receive, only what additionally gets printed as a side effect. This is the canonical template referenced in Part I §9.

16.2 A Logging Decorator

```
from functools import wraps

def logger(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper
```

Cell explanation: Structurally identical to the timer in §16.1, differing only in what observable behavior the wrapper performs: here, an f-string reports which function is being called by name, using func.__name__ — notably accessing the original function's name directly from the closure-captured func object, not from the wrapper's own (potentially confusing) name. This is the same structural template as §16.1, underscoring Part I §9's point that the timing and logging decorators are two instances of one reusable pattern, differing only in their wrapper's content rather than their overall shape.

Summary

Decorators are not an independent language feature but a direct application of three more fundamental properties already present in Python: functions are first-class objects (§1), which permits higher-order functions that accept or return them (§2); closures (§3) allow an inner wrapper function to retain a live reference to the function it wraps, even after the outer decorator function has returned; and the `@` syntax (§4) is sugar for a single, explicit reassignment of a name to the decorator's return value. Every decorator in this chapter — from the minimal Before/After example through the timing and logging templates — is an instance of one repeating structure: accept a function, define a closure-backed wrapper generalized with `*args/**kwargs`, decorate that wrapper with `@wraps` to preserve metadata, perform additional behavior around a delegating call to the original function, and return the wrapper in place of the original. Understanding this single structure is sufficient to read, write, and reason about essentially any decorator encountered in practice, including the standard library's own (`@classmethod`, `@property`, `@lru_cache`), each of which applies the identical mechanism to a specific, common need.